

# Debate técnico — Smart LED Engine

Proposta para Smart Signage PRO · Conversa de produto e arquitetura  
TotemDigital · 23 de junho de 2026

---

## Sobre este documento

Este PDF consolida a conversa realizada antes da implementação de melhorias pendentes no Studio Vx5. O utilizador apresentou uma proposta de novo módulo para o Smart Signage PRO; em seguida foi feita uma análise técnica cruzando a proposta com o estado atual do repositório TotemDigital.



### Três cenários de integração

| Tipo              | Nível de acesso                                   | Integração                                            |
|-------------------|---------------------------------------------------|-------------------------------------------------------|
| Totalmente aberta | Android acessível, ADB, instalação de APK         | Excelente — player próprio instalável                 |
| Semiaberta        | Upload e controle por protocolo, sem instalar app | Muito boa — Smart envia ficheiros e comandos via rede |
| Fechada           | Só software do fabricante                         | SDK oficial ou engenharia reversa do protocolo        |

**Estratégia sugerida na proposta:** começar por HUIDU e Xixun (mais possibilidades históricas de integração) e depois adicionar NovaStar Taurus (grande parte do mercado profissional).

# Parte 2 — Análise e debate técnico

## Leitura da proposta

A direção é coerente estrategicamente e alinha-se ao posicionamento **Smart Signage PRO** (multi-dispositivo, integrador, OOH). O diferencial é real: o mercado está fragmentado em CMS por fabricante, enquanto o TotemDigital já possui campanha → dispatcher → totem → playback maduro no Android (Player-AD).

O ponto central não é «mais um player», mas **mudar o destino do dispatch**: em vez de HDMI → TV, Ethernet → controladora LED, que já é o player.

## Estado atual do repositório TotemDigital

| Camada existente                            | Reuso para Smart LED Engine                                |
|---------------------------------------------|------------------------------------------------------------|
| totems, campanhas, dispatcher, DispatchPlan | Alto — lógica de «o que tocar» já existe                   |
| Player-AD + /api/player/*                   | Parcial — modelo pull; LED costuma ser push (FTP/HTTP/TCP) |
| smart_tvs (marca, modelo, plataforma)       | Conceitual — dispositivo físico ligado ao totem            |
| OTA por plataforma (android, webos...)      | Baixo — LED não recebe APK da mesma forma                  |
| Studio compacto                             | Nenhum — módulo é claramente PRO                           |
| NovaStar / HUIDU / Xixun no código          | Não implementado                                           |

Conclusão: o CMS e o motor de campanha existem; falta uma **camada de adaptadores de hardware** entre o dispatch e a controladora.

## Arquitetura recomendada (3 camadas)

[CMS existente] Campaign / Playlist / Mídia ↓ Dispatcher / DispatchPlan ↓ [NOVO] Smart LED Engine · Device Registry + Discovery · LED Orchestrator · Sync / Jobs / Retry ↓ [NOVO] Driver Layer NovaStar | HUIDU | Xixun | Colorlight ↓ Painéis LED na rede

## Responsabilidades do Smart LED Engine

- Receber plano já resolvido (DispatchPlan ou LEDDeployJob derivado).
- Traduzir para protocolo do fabricante (upload, playlist, schedule, brilho).
- Monitorar status (online, temperatura, falha de ficheiro, brilho).
- Reportar de volta ao heartbeat/totem para painel unificado.
- Não reinventar campanha — reutilizar o que o dispatcher já resolve.

## Cenários de integração — ordem prática

| Cenário            | Exemplo             | Estratégia                  | Esforço | Risco                |
|--------------------|---------------------|-----------------------------|---------|----------------------|
| A — Android aberto | NovaStar Taurus TBx | Instalar Player-AD via ADB  | Baixo   | Firmware fechado     |
| B — Semiaberta     | HUIDU A/C, Xixun    | Driver de rede FTP/HTTP/TCP | Médio   | Protocolo por modelo |
| C — Fechada        | Colorlight antigo   | SDK ou eng. reversa         | Alto    | Suporte contínuo     |

**Recomendação de piloto duplo:** (1) Taurus com Player-AD para provar valor em semanas; (2) um modelo HUIDU com driver nativo para provar integração sem HDMI nem APK.

## Auto-discovery — expectativa realista

| Método                    | Uso             | Limitação                      |
|---------------------------|-----------------|--------------------------------|
| UDP broadcast             | HDPlayer, Xixun | VLAN, firewall, Wi-Fi isolado  |
| mDNS/Bonjour              | Alguns IoT      | Raro em LED industrial         |
| Scan subnet + fingerprint | ViPlex, HDSet   | Lento; pode exigir credenciais |
| TCP após login            | NovaStar        | Porta e handshake específicos  |

Fase 1: scan de subnet + identificação por porta/resposta. Fase 2: mDNS onde existir + importar IP do software oficial como fallback. Senha admin da controladora continuará necessária na maioria dos casos.

## Onde Smart Signage pode superar os CMS nativos

- Um painel para TV + Android box + LED multi-marca.
- Playlist ligada a campanha, contrato, plano e mix (OOH comercial).
- Alertas no dashboard + diagnostics do dispatcher.
- Faturamento anunciante/exibidor já existente no PRO.

MVP do LED Engine não precisa: sync multi-painel frame-perfect, mapa de pixels, correção de gabarito — isso é v3+.

## Riscos antes de implementar

- Fragmentação de firmware — mesmo modelo pode falar protocolos diferentes.
- Suporte pós-venda — integrador culpa o Smart Signage; exigir logs por job.
- Engenharia reversa — risco legal em clientes enterprise; priorizar SDK oficial.
- Telemetria heterogênea — modelo canônico (brightness, temp, power, storage).
- Sync multi-painel — produto à parte (NTP, frame sync); não é MVP.
- Manter LED apenas no PRO (feature flag), como SmartDisplayFX no Studio.

## Modelo de dados (esboço conceitual)

- led\_controllers — vendor, model, firmware, ip, driver\_id, capabilities.
- led\_deploy\_jobs — totem\_id, dispatch\_snapshot, status, last\_error.
- led\_telemetry — brightness, temp, disk, last\_seen.
- Totem continua entidade de negócio no dispatcher; controladora é endpoint de execução.

## Roadmap sugerido no debate

### MVP (8–12 semanas, 1 fabricante + 1 modelo)

- Cadastro manual (IP + credencial).
- Upload vídeo/imagem via driver HUIDU ou Xixun.
- Playlist simples (ordem + duração).
- Status online/offline + último deploy.
- UI PRO: Locais → Painéis LED.

### v2

- Auto-discovery na LAN.
- Brilho remoto + agendamento.
- Segundo fabricante (NovaStar protocolo ou Taurus + Player-AD).
- Integração dispatcher: campanha ativa → deploy automático.

### v3

- Telemetria e alertas avançados.
- Sync multi-painel básico.
- Colorlight / processadores fechados via SDK.

## Perguntas em aberto (antes de codar)

- Público piloto: integrador HUIDU no Brasil ou cliente Taurus?
- Servidor na mesma LAN do painel (on-prem) ou cloud com VPN?
- Substituir HDPlayer/ViPlex ou conviver em paralelo?
- Em Android aberto: sempre Player-AD ou driver nativo mais leve?
- MVP só playback de campanha ou já brilho + monitoramento?

## Veredito da análise

A proposta é **coerente, diferenciada e alinhada ao PRO**. Não se recomenda começar por auto-discovery + quatro drivers + sync multi-painel em paralelo. Caminho mais seguro: (1) provar valor com uma controladora real; (2) ligar ao dispatch existente; (3) escalar drivers e discovery depois.

Próximo passo sugerido na conversa: debater qual fabricante/modelo usar como piloto e desenhar o contrato do driver (discover | connect | upload | setPlaylist | setBrightness | getStatus) sem implementação imediata.